

Skill-Based Architecture for Reusable, AI-Native Enterprise Modules: An Empirical Study

Nikhil Jathar
AvanSaber, Inc.
San Antonio, TX, USA
nikhil.jathar@avansaber.com

Abstract—We present a skill-based architecture pattern for building reusable, AI-native enterprise software modules. Each business domain (accounting, inventory, payroll) is an independent executable with a declarative manifest file, sharing a single embedded database through a read-many/write-one isolation rule. A large language model (LLM) runtime reads these manifests and translates natural language into structured commands, requiring zero per-module customization. We validate this pattern through a production system comprising 47 modules with 2,586 actions across 688 tables, deployed across 14 industry verticals with no vertical-specific runtime changes. The same architecture, the same AI runtime, and the same manifest format serve healthcare, education, construction, legal, and ten other domains without modification. We report empirical results: 7,627 automated tests, 23 accounting invariant checks, sub-20ms read latency under concurrent write load, and setup time under 5 minutes on commodity hardware. A self-improving layer demonstrates that AI agents can generate new domain modules autonomously, with 144/144 tests passing and zero violations of financial integrity constraints. The system has been in production since March 2026 with 100+ active users.

Index Terms—software reuse, skill-based architecture, AI-native enterprise software, information integration, modular systems, embedded databases, ERP

I. INTRODUCTION

Enterprise Resource Planning (ERP) systems unify a company’s financial ledger, supply chain, payroll, and customer management into a single system of record. Yet deployment remains painful. Panorama Consulting’s 2024 ERP Report found a median ERP implementation timeline of 15.5 months and a median project cost of \$450,000 [1]. Even open-source alternatives like ERPNext require a multi-service stack (MariaDB, Redis, and Node.js, plus Nginx and Supervisor in production) [2] and, in our experience, one to two weeks of configuration before a small business can book its first invoice. Research by the Consortium for Information and Software Quality estimates that poor software quality cost the US economy at least \$2.41 trillion in 2022 [3].

Meanwhile, software engineering is shifting toward LLM-centered workflows. LLMs translate natural language into executable code [4], reach competition-level programming performance [5], and measurably improve developer productivity [6]. Recent work builds LLM agents that plan, implement, and repair software with growing autonomy [7], [8], [9]. Enterprise software has not internalized this shift architecturally: AI is bolted onto monolithic ERPs as an assistant add-on rather than serving as the system’s interface layer.

The problem is architectural. Traditional ERPs are monolithic web applications. Each module is tightly coupled to a shared ORM, a web framework, and a rendering pipeline. Adding a new industry vertical (healthcare, construction, legal) requires deep framework knowledge and months of integration work. Reuse across verticals is limited to shared utility functions; the architecture itself does not enforce or facilitate reuse.

We argue that modular skill-based architectures, combined with LLM interface layers, enable a fundamentally different model for enterprise software, one in which reuse is a structural property, not an afterthought. Instead of a monolithic application, an ERP is composed of independent domain modules (“skills”) that share a single database and are operated through natural language. The LLM reads a declarative manifest (the SKILL.md file), extracts parameters from the user’s request, invokes the module as a subprocess, and formats the response. This cycle is domain-agnostic: the same runtime that operates an accounting module operates a healthcare module, a construction module, or a legal module, with zero per-domain customization.

This paper makes four contributions:

- (a) **A reusable architecture pattern.** We define skill-based architecture: independent executables sharing a single embedded database through a read-many/write-one rule, communicating via subprocess invocation, with a declarative manifest serving as the universal reuse contract.
- (b) **Empirical validation at scale.** We present a production system built as 47 skills with 2,586 actions, 688 tables, and 7,627 automated tests across 14 industry verticals, all running on one architecture with one AI runtime. The system has been in production since March 2026.
- (c) **A self-improving module layer.** We describe a platform where AI agents generate new domain modules autonomously, governed by 18 constitutional articles that protect financial integrity. Three generated modules passed 144/144 tests with zero direct ledger writes.
- (d) **Quantitative comparison.** We benchmark setup time, cost, and architecture against representative enterprise and open-source ERP platforms, and report SQLite stress test results validating embedded database suitability for enterprise workloads.

The remainder of this paper is organized as follows. Section

II describes the architecture. Section III presents the case study, including scope, testing, and deployment results. Section IV covers the self-improving module layer. Section V compares with existing systems. Section VI examines LLM interpretation reliability and failure modes. Section VII discusses limitations and future work, and Section VIII concludes.

II. ARCHITECTURE

A. Overview and Manifest Format

The skill-based architecture decomposes enterprise software into independent domain modules. Each module is a self-contained directory with four components: a SKILL.md manifest, one or more executable scripts, optional reference documents, and tests. The SKILL.md file uses YAML frontmatter for machine-readable metadata (name, version, actions with typed parameters) and Markdown body for human-readable guidance (usage examples, domain knowledge, error handling) [10]. A typical manifest is under 300 lines. The manifest plays the role that declarative interface specifications such as OpenAPI [11] play for web services (a machine-readable contract) while additionally carrying the human-readable domain guidance an LLM needs to operate the module. The decomposition itself follows the classic information-hiding principle [12]: each module hides its implementation behind a declared interface.

The manifest serves three roles simultaneously: documentation for developers, interface specification for the AI runtime, and reuse contract for the platform. When a user sends a natural language request, the runtime (1) selects the relevant module from activation triggers in the manifest, (2) extracts structured parameters, (3) constructs a subprocess call to the module's script, (4) executes it, and (5) formats the JSON response for the user. This cycle requires no per-module training, fine-tuning, or prompt engineering. The AI derives all behavior from the manifest. In production, the runtime has been operated with several commercial LLM services: Anthropic Claude (Opus 4.8), Moonshot AI Kimi (K2.6), Google Gemini (3.5 Flash), and OpenAI Codex (GPT-5.5), all accessed through the OpenClaw agent platform [10]. The architecture is model-agnostic; it requires only an LLM capable of structured tool invocation.

This approach differs from three common modular architectures. Microservices [13], [14] communicate via network protocols, introducing service meshes, API gateways, and distributed tracing. Skills are local executables with no network overhead between modules. Plugin systems (e.g., WordPress, VS Code) share a host process memory space; a misbehaving plugin can corrupt another's state. Skills run as independent processes with no shared memory. Traditional ERP modules (Odoo apps, ERPNext modules) share an ORM and web framework, and customizing them spans a well-documented spectrum of increasingly invasive tailoring choices [15]; skills share only a database schema and a set of conventions.

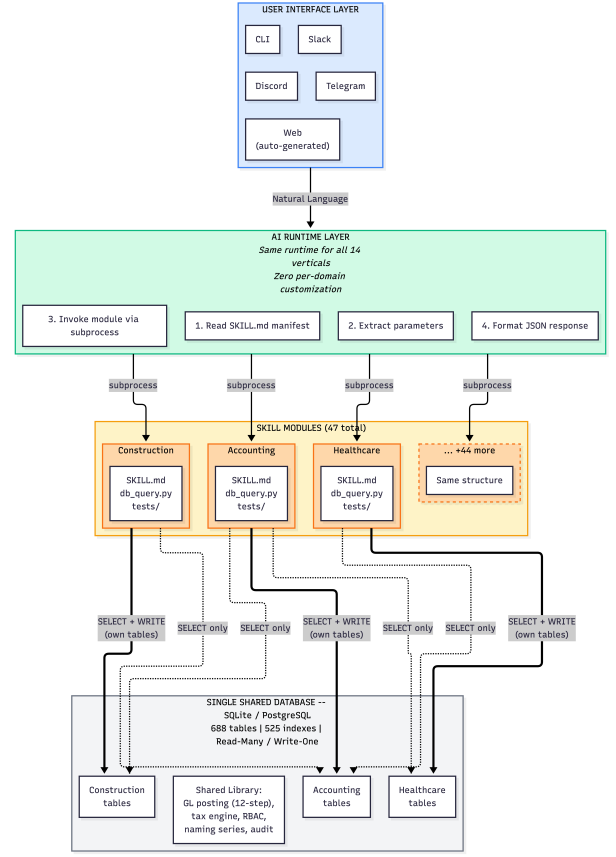


Fig. 1. Skill-based architecture. Each business module is an independent executable with a declarative manifest (SKILL.md), sharing a single embedded database through a read-many/write-one isolation rule. Solid arrows indicate write access (own tables only); dashed arrows indicate read-only access (any table).

B. Single Shared Database

All modules share a single SQLite database file. This provides immediate cross-domain visibility (the reports module can join selling and payments tables in one SQL statement), eliminates data synchronization problems, and guarantees referential integrity through foreign keys. The current schema comprises 688 tables and 525 indexes across 47 modules.

SQLite was chosen for three reasons: zero infrastructure (no database server), single-file deployment (backup with a file copy), and performance that matches server databases for single-company access patterns (sequential writes, concurrent reads, sub-million row tables) [16]. All query construction uses a PyPika abstraction layer, making the system database-agnostic. PostgreSQL compatibility has been verified end-to-end (186 tables created, CRUD operations functional, GL entries balanced).

C. Skill Isolation: Read-Many, Write-One

Each table has a single owning module. Only that module may INSERT, UPDATE, or DELETE rows. Any module may SELECT from any table. This rule delivers three benefits: (1) clear ownership, since a bug in sales_invoice must be in the selling module; (2) safe schema evolution, since a module can add columns to its tables without coordinating with other

modules as long as new columns have DEFAULT values; and (3) audit clarity, since every write traces to a single module.

D. Data Integrity and Transactions

Financial amounts are stored as TEXT and processed as Python Decimal objects with ROUND_HALF_UP, avoiding IEEE 754 floating-point representation errors. All identifiers are UUID4 hex strings, eliminating auto-increment contention. All database connections enforce three PRAGMAs: journal_mode = WAL (concurrent reads during writes), foreign_keys = ON (referential integrity), and busy_timeout = 5000 (graceful lock contention).

When one module needs to trigger behavior in another's domain, it uses subprocess invocation. This maintains isolation (the caller never imports the callee's code) and uses the same interface as the AI layer. Process creation overhead is approximately 50ms per call; the most complex workflow involves at most 6 cross-module calls, making this overhead negligible.

Every submit operation runs in a single database transaction. When a user submits a sales invoice, the status update, GL entry creation, payment ledger entry, stock ledger entry, naming series increment, and audit log insertion all occur within one BEGIN...COMMIT block. If any step fails, the entire transaction rolls back. There are no partial postings.

III. CASE STUDY: A 47-MODULE PRODUCTION ERP

A. Scope and Structure

We built ERPCLaw, a production ERP system comprising 47 skill-based modules spanning 14 industry verticals:

Each module follows an identical structure: a SKILL.md manifest, a main script (all actions routed through an --action flag), optional references, and a tests directory. A shared library (33 Python modules, 11,007 lines) provides cross-cutting functionality: GL posting with 12-step validation, stock ledger entry creation with FIFO and weighted-average valuation, tax computation, database-agnostic query abstraction, naming series generation, RBAC enforcement, and inter-module subprocess invocation.

The reuse point is critical: the healthcare module, the legal module, and the construction module all share the same GL posting library, the same tax engine, the same naming series generator, and the same AI runtime, with zero per-vertical customization.

B. Cross-Module Workflow: Order-to-Cash

The order-to-cash workflow demonstrates cross-module integration. A user says: "Create a quotation for 10 widgets at \$3,000 each." The system orchestrates a 10-step sequence across the selling and payments modules: quotation creation and submission, sales order derivation, delivery note with stock ledger entries, sales invoice with GL entries (debit AR \$30,000, credit Revenue \$30,000), and payment receipt with GL entries (debit Bank \$30,000, credit AR \$30,000). At completion, the trial balance reflects balanced Revenue, COGS, Bank, and zero outstanding receivables. Every GL

posting passes 12 validation checks, and the invariant engine verifies ledger integrity after each step.

C. Testing

The system uses 7,627 automated tests across four layers:

The most distinctive element is the accounting invariant engine: 23 structural checks that run automatically after every test touching the general ledger, implemented as a pytest autouse fixture. The checks span five severity levels: critical (global double-entry balance, per-voucher balance, GL immutability), high (cancellation symmetry, orphan detection), medium (naming series monotonicity, status-GL consistency), structural (FK integrity, trial balance zero, atomic transactions), and extended (cross-company isolation, payment allocation bounds, tax coherence). The engine adds under 50ms per test and caught 5 bugs during development that would have passed conventional unit tests.

D. Development Timeline

A solo developer built the system using an AI coding assistant over approximately 4 weeks (February 15 to March 17, 2026). This velocity reflects the architecture, not the developer, and is consistent with measured productivity effects of AI coding assistance [6]. Each module's script is 500 to 2,000 lines of self-contained Python that reads arguments and returns JSON. There is no framework to learn, no ORM to configure, no web routes to register. The final codebase comprises 200,207 lines of production Python and 83,745 lines of test Python.

E. Production Deployment and Performance

The system runs on a single Ubuntu 24.04 server (4 vCPUs, 15 GB RAM, approximately \$40/month). There is no database server, no Redis, no message queue. The system has been deployed in production since March 2026 and is actively used by over 100 users across multiple industry verticals. This is not a research prototype; it is shipping software processing real business transactions. The system is publicly accessible at <https://www.erpclaw.ai>.

Setup from zero to a functional ERP takes under 5 minutes. Table III situates this against reported industry figures. The commercial-system figures are implementation-industry estimates [1], [20], [21], not controlled measurements; they characterize the deployment-effort gap rather than rank vendors.

*Authors' estimate from self-hosted deployment experience with the documented multi-service stack [2].

SQLite stress tests confirm suitability for enterprise workloads: 5 concurrent writers with 0 lock errors, average 19ms read latency under write load, and constant-time list queries (0.6ms at both 100K and 1M rows). A six-gate deployment validation confirmed production readiness: 7,627 tests passing, 12/12 verticals operational, 6/6 cross-module integrations verified, and a final GL balance of \$1,060,433 in perfect double-entry equilibrium.

TABLE I
SYSTEM SCOPE

Core ERP	1	396	Setup, GL, selling, buying, inventory, payments, tax, HR, payroll, reports, billing
Addons	16	625	CRM, manufacturing, projects, assets, quality, compliance, logistics, POS, treasury
Regional	4	111	Tax/payroll overlays: India, Canada, UK, EU
Verticals	26	1,454	Healthcare, education, property, retail, construction, agriculture, automotive, food service, hospitality, legal, nonprofit, + 3 AI-generated
Total	47	2,586	688 tables, 200,207 lines Python

TABLE II
TESTING PYRAMID

L0: Constitutional	153	18 constitutional article enforcement
L1: Unit/Integration	4,841	Business logic, GL posting, validation
L2: Contract (auto-generated)	2,606	Schema and interface contracts for all modules
L3: Smoke	27	Cross-module business flows
Total	7,627	

TABLE III
SETUP TIME AND COST CONTEXT

ERP industry median [1]	15.5 months	\$450K
Oracle NetSuite [20]	3-12 months	\$30K-150K+
Odoo Enterprise [21]	1-5 months	\$10K-50K
ERPNext (self-hosted)*	1-2 weeks	\$5K-15K
ERPCLaw (ours)	5 minutes	\$480

IV. SELF-IMPROVING MODULES

The 44 published modules encode business domain knowledge in executable form. An AI agent with access to these modules can extrapolate to any business type, because the patterns repeat: every business has customers, invoices, payments, and a general ledger. The self-improving platform layer enables AI agents to generate, test, and deploy new modules, but only within the constraints of a formal constitution.

The Constitution. 18 machine-readable articles in four categories govern all generated modules. Financial integrity articles (1-6) enforce double-entry balance, GL immutability, money-as-text, referential integrity, RBAC, and append-only audit trails. Operational safety articles (7-12) enforce transaction atomicity, naming series uniqueness, fiscal period enforcement, and cancellation-by-reversal. Structural articles (13-16) enforce read-many/write-one ownership, UUID keys, manifest compliance, and status-GL consistency. Meta-rules (17-18) prevent any generated module from weakening the constitution and require automatic verification.

Enforcement and Audit. Each article is a machine-evaluable record: an identifier, a condition evaluable on statically extracted properties of candidate code or schema, an enforcement type (static analysis or runtime verification), a severity level, and a bypass policy. Enforcement is deterministic rather than prompt-based. A validator parses each generated module’s code with Python’s AST parser, identifies every database operation and its target tables, and blocks deployment of any module that writes the protected ledger tables (general ledger, stock ledger) directly, enforcing Article

TABLE IV
AI-GENERATED MODULES

Grooming	Pet grooming salons	6	13	48/48 pass
Tattoo	Tattoo parlors	8	18	48/48 pass
Storage	Self-storage facilities	9	17	48/48 pass

6: all GL postings must route through the shared posting library and its 12-step validation pipeline. Critical-severity violations block deployment, with no autonomous or human bypass. The 153 L0 constitutional tests verify enforcement continuously, and every enforcement decision is recorded in the append-only audit trail. The validator, the article registry, and the deployment pipeline are themselves on an immutable exclusion list that generated code cannot modify, so the AI cannot weaken the machinery that constrains it.

Empirical Result. Three proof-of-concept modules were generated:

Generation required only 19 human decisions across all three modules: all business preference choices, not accounting decisions. Zero direct GL writes were generated; all financial operations route through the shared posting library. This validates the 80/15/5 thesis: 80% of module creation is mechanical, 15% is pattern-matching from existing modules, and 5% requires human judgment on domain preferences.

These three domains were deliberately low-complexity: appointment-driven service businesses with standard invoicing and no regulatory overlay. They validate the constitutional mechanism (AI-generated code that cannot violate financial integrity) rather than generation quality in domains with deep regulatory or workflow complexity, such as clinical compliance or multi-jurisdiction payroll. We bound the claim accordingly: the self-improving layer demonstrates safe autonomous generation in simple domains; validation in complex domains remains future work (Section VII).

Seven Inviolable Laws. The platform enforces seven structural properties that no generated module can override: (1) double-entry balance on every transaction, (2) GL and stock ledger immutability, (3) UUID identity with naming series, (4) money stored as TEXT and processed as Decimal, (5) foreign key constraints at the database level, (6) RBAC on every write, and (7) append-only audit trail with SHA-256 chain checksums.

V. COMPARISON WITH EXISTING SYSTEMS

Table V contrasts design choices across deployment contexts. The systems target fundamentally different segments:

ERPclaw is built for small and medium businesses operating a single organization, while SAP S/4HANA and NetSuite serve large multi-entity enterprises. The table therefore characterizes positioning, not head-to-head performance.

Odoo and ERPNext, as modular monoliths, share the traditional pattern: adding a new vertical requires learning the framework’s ORM, web routing, form rendering, and build pipeline, which is the invasive end of the ERP tailoring spectrum [15]. In skill-based architecture, a new vertical requires a SKILL.md manifest and a Python script that returns JSON. The AI runtime, web UI auto-generation, testing harness, and deployment pipeline are all reused without modification.

Traditional ERPs hold advantages in three areas: multi-tenancy (shared infrastructure with tenant isolation), battle-tested scale (Fortune 500 supply chains), and compliance certifications (SOC 2, ISO 27001). The skill-based approach wins on deployment speed, cost, data sovereignty, and extensibility. The LLM interface eliminates an entire category of engineering (dashboard design, form rendering, navigation, user training) by replacing it with a declarative manifest that the AI runtime interprets directly.

VI. LLM INTERPRETATION RELIABILITY

The architecture places an LLM between the user and every module. What happens when the model misinterprets? LLMs are known to produce fluent but incorrect output [17], and an enterprise system cannot assume its runtime is always accurate. Our position is that interpretation reliability must be engineered around the model, not assumed of it.

Containment by construction. The LLM never executes SQL, never calls module internals, and never writes to the database. Its only output channel is a subprocess invocation of an action declared in a manifest, with typed parameters. A hallucinated action name or malformed parameter set fails validation inside the module before any state change occurs, on the same validation path that human-facing interfaces use. This differs from prompt-level guardrails [18] in that enforcement happens deterministically in code downstream of the model; it complements evidence that constraining LLM output to structured formats is practical at industrial scale [19].

Failure modes. We distinguish three classes. First, wrong module selection on an ambiguous request: the runtime picks a plausible but unintended module. Second, wrong parameter extraction: the right action is invoked with incorrect values. Third, valid-but-unintended operations: a syntactically and semantically valid command that does not match user intent. The first two classes typically fail loudly: parameter validation rejects missing or type-inconsistent values, and business rules reject semantically invalid operations (posting to a closed fiscal period, referencing an unknown customer, negative stock quantities). The third class is the hardest, because the operation is legitimate and no validation layer can catch it. The system bounds its blast radius structurally. Documents follow a draft-then-submit lifecycle, so financial postings occur only at an explicit submit step that gives the user a review point. Every submit is a single atomic transaction that rolls back

completely on failure. The 12-step GL validation and 23 invariant checks guarantee ledger integrity regardless of intent. And the append-only audit trail with cancellation-by-reversal means any unintended posting can be identified and reversed without destructive correction.

Generated code. For the self-improving layer, the analogous risk is a generating agent emitting harmful code rather than a runtime misreading a request. Static pattern matching alone is a weak defense here: destructive logic can be obfuscated through dynamic string construction that no fixed pattern catches. Enforcement therefore relies on deterministic AST analysis of every database operation in generated code, the constitutional test suite, and human review gates for higher-risk operations, with the enforcement machinery itself on an immutable exclusion list the agent cannot modify (Section IV).

Scope of the claim. Our end-to-end tests use developer-authored, unambiguous requests. Production users issue vague, incomplete, and contradictory requests, and we have not yet quantified interpretation accuracy under ambiguity at production scale (Section VII). The reliability claim is therefore bounded: the architecture guarantees that a misinterpreted request cannot bypass declared actions or violate ledger invariants; it does not guarantee that every interpretation is correct.

VII. LIMITATIONS AND FUTURE WORK

Three limitations are architectural, not merely a matter of development time.

SQLite concurrency. SQLite permits one writer at a time. WAL mode allows concurrent readers, but true write concurrency is not possible. Five concurrent writers succeed with zero lock errors due to busy timeout, but execute sequentially. The PyPika migration has made PostgreSQL available as an alternative; switching requires a configuration change, not a code change.

Single-tenant architecture. Each deployment serves one organization. Multi-tenant SaaS is not supported. A multi-tenant gateway is architecturally feasible but would add complexity around provisioning, isolation, and billing.

Ecosystem maturity. SAP has 25,000+ partners. Our system’s ecosystem is nascent. Four regional overlays cover basic compliance, but tax law changes constantly.

Future directions include testing the self-improving layer at scale and in regulation-heavy domains (Section IV), building a multi-tenant gateway, expanding database backend support, and quantifying LLM interpretation accuracy under ambiguous real-world requests in production (Section VI).

VIII. CONCLUSION

This paper has presented skill-based architecture as a pattern for building reusable, AI-native enterprise modules and validated it through a 47-module production system spanning 14 industry verticals. The core reuse claim is empirical: one architecture, one AI runtime, and one manifest format serving healthcare, education, construction, legal, retail, and nine other domains with zero per-vertical customization.

TABLE V
DEPLOYMENT-CONTEXT COMPARISON

Target segment	SMB, single organization	Large enterprise	Mid-size to large enterprise
Architecture	Skill-based (47 modules)	Monolithic	Cloud monolith
Database	SQLite or PostgreSQL	SAP HANA	Oracle DB
Tenancy	Single-tenant	Multi-tenant	Multi-tenant
Interface	Chat + Web	Web dashboard	Web dashboard
Compliance certifications	None yet	SOC 2, ISO 27001	SOC 2, ISO 27001
AI-native	Yes (LLM reads manifest)	AI add-on (BTP)	AI add-on
Self-improving	Yes (AI-generated modules)	No	No
Setup time and cost	Minutes; \$480/yr	Months+; \$100Ks+ [1]	3-12 months; \$30K-150K+ [20]

Three quantitative results support the pattern’s viability. First, 7,627 automated tests with 23 accounting invariants provide continuous structural assurance across all 47 modules. Second, SQLite handles 1 million GL entries with 0.6ms indexed lookups and 3.1s aggregate queries, with PostgreSQL available for write concurrency. Third, the self-improving layer generated three new modules with 144/144 tests passing and only 19 human decisions, all of them domain preferences rather than accounting logic.

The system has been in production since March 2026 with over 100 active users. It runs on a single \$40/month server with a 5-minute setup time. We do not claim this architecture replaces enterprise systems for multinational corporations. But for small and medium businesses that need financial software they can own and operate without a consultant, the combination of modular reuse, embedded databases, subprocess isolation, and AI-native interfaces represents a viable alternative to the monolithic model that has dominated enterprise software for three decades.

ACKNOWLEDGMENT

ERPCLaw was developed with the assistance of Claude Code (Anthropic). The architectural patterns draw on extensive study of ERPNext (Frappe Technologies), Odoo (Odoo SA), immutable ledger designs from Square Books and Modern Treasury, and Lago (open-source billing). We thank the OpenClaw team for the platform that makes skill-based architecture possible.

REFERENCES

- [1] Panorama Consulting Group, “The 2024 ERP Report,” Denver, CO, USA, 2024. [Online]. Available: <https://www.panorama-consulting.com/resource-center/erp-report-archives/>
- [2] Frappe Technologies, “Installation,” Frappe Framework Documentation, 2025. [Online]. Available: <https://docs.frappe.io/framework/user/en/installation> (accessed Jun. 9, 2026).
- [3] H. Krasner, “The Cost of Poor Software Quality in the US: A 2022 Report,” Consortium for Information and Software Quality (CISQ), Dec. 2022. [Online]. Available: <https://www.it-cisq.org/wp-content/uploads/sites/6/2022/11/CPSQ-Report-Nov-22-2.pdf>
- [4] M. Chen et al., “Evaluating large language models trained on code,” 2021, arXiv:2107.03374.
- [5] Y. Li et al., “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, Dec. 2022, doi: 10.1126/science.abq1158.
- [6] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Aftandilian, “Measuring GitHub Copilot’s impact on productivity,” *Commun. ACM*, vol. 67, no. 3, pp. 54–63, Mar. 2024, doi: 10.1145/3633453.
- [7] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, Art. 220, Dec. 2024, doi: 10.1145/3695988.
- [8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, 2024.
- [9] C. Qian et al., “ChatDev: Communicative agents for software development,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Bangkok, Thailand, 2024, pp. 15174–15186, doi: 10.18653/v1/2024.acl-long.810.
- [10] OpenClaw, “Skills,” OpenClaw Documentation, 2026. [Online]. Available: <https://docs.openclaw.ai/tools/skills> (accessed Jun. 9, 2026).
- [11] OpenAPI Initiative, “OpenAPI Specification v3.1.1,” Linux Foundation, Oct. 2024. [Online]. Available: <https://spec.openapis.org/oas/v3.1.1.html>
- [12] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Commun. ACM*, vol. 15, no. 12, pp. 1053–1058, Dec. 1972, doi: 10.1145/361598.361623.
- [13] J. Lewis and M. Fowler, “Microservices: A definition of this new architectural term,” *martinfowler.com*, Mar. 25, 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [14] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2021.
- [15] L. Brehm, A. Heinzl, and M. L. Markus, “Tailoring ERP systems: A spectrum of choices and their implications,” in *Proc. 34th Annu. Hawaii Int. Conf. System Sciences (HICSS)*, Maui, HI, USA, 2001, doi: 10.1109/HICSS.2001.927130.
- [16] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. Kennedy, and J. M. Patel, “SQLite: Past, present, and future,” *Proc. VLDB Endowment*, vol. 15, no. 12, pp. 3535–3547, 2022, doi: 10.14778/3554821.3554842.
- [17] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Comput. Surv.*, vol. 55, no. 12, Art. 248, Mar. 2023, doi: 10.1145/3571730.
- [18] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” in *Proc. 2023 Conf. Empirical Methods Natural Language Processing: System Demonstrations*, Singapore, 2023, pp. 431–445, doi: 10.18653/v1/2023.emnlp-demo.40.
- [19] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen, “Let me speak freely? A study on the impact of format restrictions on large language model performance,” in *Proc. 2024 Conf. Empirical Methods Natural Language Processing: Industry Track*, Miami, FL, USA, 2024, pp. 1218–1236, doi: 10.18653/v1/2024.emnlp-industry.91.
- [20] Numeric, “NetSuite implementation cost: What to expect,” May 27, 2025. [Online]. Available: <https://www.numeric.io/blog/netsuite-implementation-cost> (accessed Jun. 9, 2026).
- [21] Aglowid IT Solutions, “Odoo implementation cost guide,” 2025. [Online]. Available: <https://aglowiditsolutions.com/blog/odoo-implementation-cost/> (accessed Jun. 9, 2026).